

Linux Class 9 - Task

sed and grep, awk Columns, Compression and Replacing Text

Submitted by: Shivakumar Muradi

Date: 01 October 2026

For this task I made a folder called class9. I copied employees.txt into it from my last class, and then typed three more small files in vi. notes.txt has seven lines and the word linux appears once on each line. line.txt has only one line, with the word linux written seven times on it. mixed.txt has the same word written as Linux, LINUX and linux, for the last question.

1. What is the difference between the sed and grep commands?

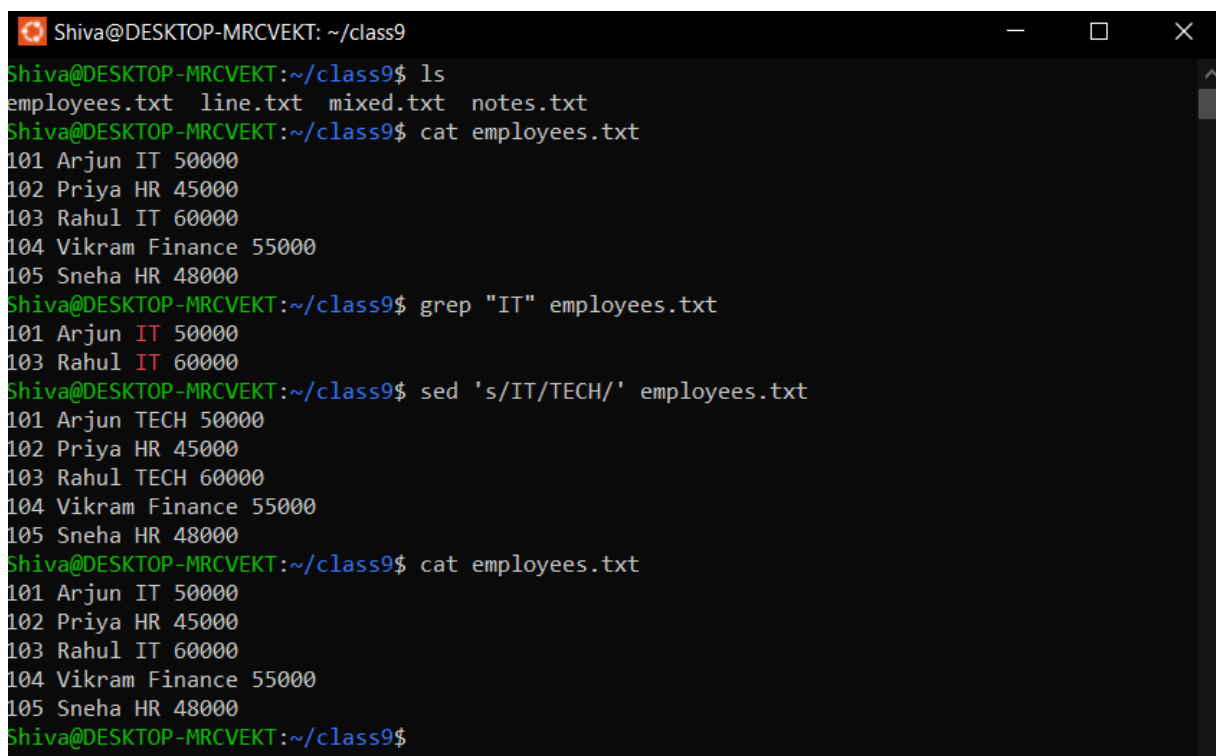
Both of them look through a file for a pattern, but what they do after finding it is completely different. grep only shows me the lines that have the word in them. sed changes the word and shows me the whole file with the change made.

```
grep "IT" employees.txt
```

This printed only the two lines that have IT in them. The other three lines were not shown at all.

```
sed 's/IT/TECH/' employees.txt
```

This printed all five lines, and in the two lines that had IT, the word became TECH. The s at the start stands for substitute, which means replace.



```
Shiva@DESKTOP-MRCVEKT: ~/class9
Shiva@DESKTOP-MRCVEKT:~/class9$ ls
employees.txt line.txt mixed.txt notes.txt
Shiva@DESKTOP-MRCVEKT:~/class9$ cat employees.txt
101 Arjun IT 50000
102 Priya HR 45000
103 Rahul IT 60000
104 Vikram Finance 55000
105 Sneha HR 48000
Shiva@DESKTOP-MRCVEKT:~/class9$ grep "IT" employees.txt
101 Arjun IT 50000
103 Rahul IT 60000
Shiva@DESKTOP-MRCVEKT:~/class9$ sed 's/IT/TECH/' employees.txt
101 Arjun TECH 50000
102 Priya HR 45000
103 Rahul TECH 60000
104 Vikram Finance 55000
105 Sneha HR 48000
Shiva@DESKTOP-MRCVEKT:~/class9$ cat employees.txt
101 Arjun IT 50000
102 Priya HR 45000
103 Rahul IT 60000
104 Vikram Finance 55000
105 Sneha HR 48000
Shiva@DESKTOP-MRCVEKT:~/class9$
```

Screenshot 1: grep shows only the matching lines, sed shows the whole file with the change.

The last command in the screenshot is important. I ran cat employees.txt again after sed, and the file still says IT. That is because sed only prints the changed version on the screen, it does not touch the real file unless I add the -i option.

This is how I remember the two:

	grep	sed
What it is for	Finding lines.	Changing lines.
What it prints	Only the lines that match.	Every line, with the changes made.
Does it change the file	Never.	Only if I add the -i option.
Main job word	search	substitute
Typical command	grep "IT" employees.txt	sed 's/IT/TECH/' employees.txt

One more thing RK sir said. grep is only for searching, but sed can do a lot more than replacing, like deleting lines with sed '4d' or printing a range of lines. grep cannot change anything at all.

2. How to display the line number followed by the last 2 columns of a file using awk?

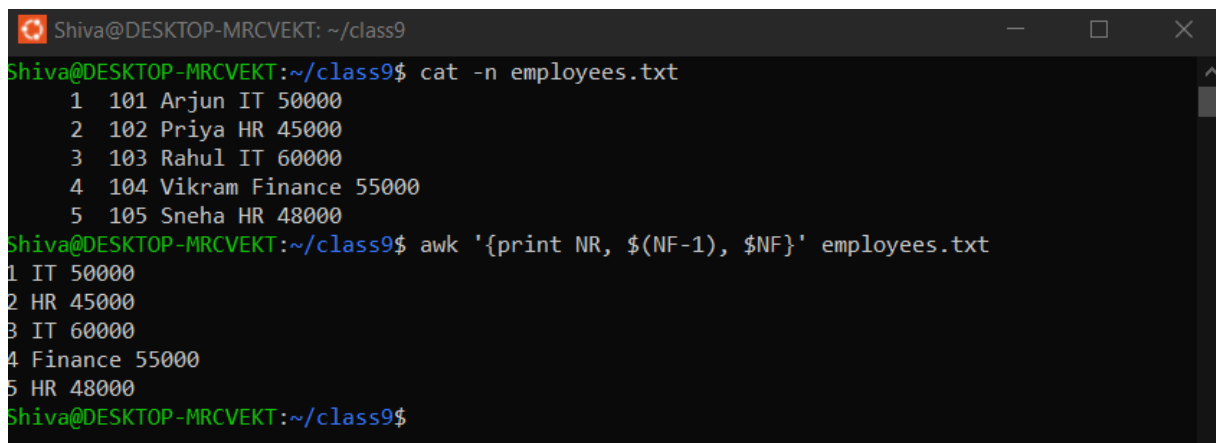
The command is:

```
awk '{print NR, $(NF-1), $NF}' employees.txt
```

Three special words do the work here:

- NR is the line number awk is on. It has no dollar sign because it is a number, not a column.
- NF is how many columns the line has, so \$NF is the last column.
- \$(NF-1) is the column just before the last one.

The commas between them are what put spaces in the output. My employees file has four columns, so the last two are the department and the salary, and the answer came out as 1 IT 50000, 2 HR 45000 and so on.



```

Shiva@DESKTOP-MRCVEKT: ~/class9
Shiva@DESKTOP-MRCVEKT:~/class9$ cat -n employees.txt
 1 101 Arjun IT 50000
 2 102 Priya HR 45000
 3 103 Rahul IT 60000
 4 104 Vikram Finance 55000
 5 105 Sneha HR 48000
Shiva@DESKTOP-MRCVEKT:~/class9$ awk '{print NR, $(NF-1), $NF}' employees.txt
1 IT 50000
2 HR 45000
3 IT 60000
4 Finance 55000
5 HR 48000
Shiva@DESKTOP-MRCVEKT:~/class9$

```

Screenshot 2: the file with line numbers, and then the same line numbers with only the last two columns.

The brackets in \$(NF-1) must not be left out. If I write \$NF-1 then awk takes the value of the last column and subtracts 1 from it, so I would get 49999 instead of IT. The brackets tell awk to work out NF-1 first and then go to that column.

I also checked it with cat -n first, because cat -n puts the real line numbers in front, so I could see that the numbers awk printed were correct.

3. How to compress multiple files?

The usual way is the tar command. tar puts many files together into one file, and with the z option it also compresses it:

```
tar -czvf allfiles.tar.gz employees.txt notes.txt mixed.txt
```

The archive name comes first and then the list of files I want inside it. The letters after the dash each do one job:

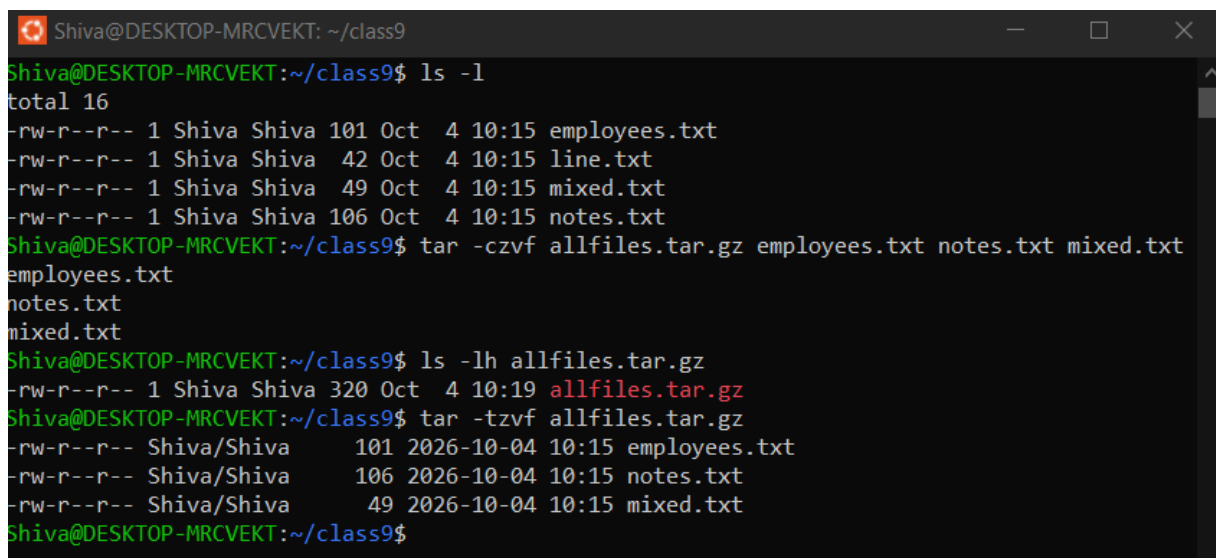
Letter	What it means
c	Create a new archive.
z	Compress it with gzip, which is why the name ends in .gz.
v	Verbose, so it prints the name of each file as it goes in.
f	The next word is the file name of the archive. This one must come last.
t	List what is inside, without taking anything out.
x	Extract, that is take the files back out.

To see what is inside without unpacking it:

```
tar -tzvf allfiles.tar.gz
```

And to get the files back out again:

```
tar -xzvf allfiles.tar.gz
```

A screenshot of a terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/class9'. The terminal shows the following commands and output:
1. `ls -l` output:

```
total 16  
-rw-r--r-- 1 Shiva Shiva 101 Oct  4 10:15 employees.txt  
-rw-r--r-- 1 Shiva Shiva  42 Oct  4 10:15 line.txt  
-rw-r--r-- 1 Shiva Shiva  49 Oct  4 10:15 mixed.txt  
-rw-r--r-- 1 Shiva Shiva 106 Oct  4 10:15 notes.txt
```


2. `tar -czvf allfiles.tar.gz employees.txt notes.txt mixed.txt` output:

```
employees.txt  
notes.txt  
mixed.txt
```


3. `ls -lh allfiles.tar.gz` output:

```
-rw-r--r-- 1 Shiva Shiva 320 Oct  4 10:19 allfiles.tar.gz
```


4. `tar -tzvf allfiles.tar.gz` output:

```
-rw-r--r-- Shiva/Shiva  101 2026-10-04 10:15 employees.txt  
-rw-r--r-- Shiva/Shiva  106 2026-10-04 10:15 notes.txt  
-rw-r--r-- Shiva/Shiva   49 2026-10-04 10:15 mixed.txt
```

Screenshot 3: three files going into one archive, and then the list of what is inside it.

One thing I noticed is that my archive came out as 320 bytes while my three files together were only about 250 bytes. That looks wrong at first, but it is because a tar file keeps a small amount of extra information about every file inside it, like its name and its permissions. On tiny files that extra information is bigger than the saving. On real files, which are much larger, it does make them much smaller.

Two other ways to do the same job:

- `zip allfiles.zip employees.txt notes.txt mixed.txt` makes a .zip file, which Windows can also open. `unzip allfiles.zip` opens it again.
- `gzip employees.txt notes.txt` compresses each file on its own instead of putting them together, and it replaces the originals with .gz files. `gunzip` brings them back.

So gzip squeezes files but does not join them, tar joins files and with -z also squeezes them, and zip does both in one format.

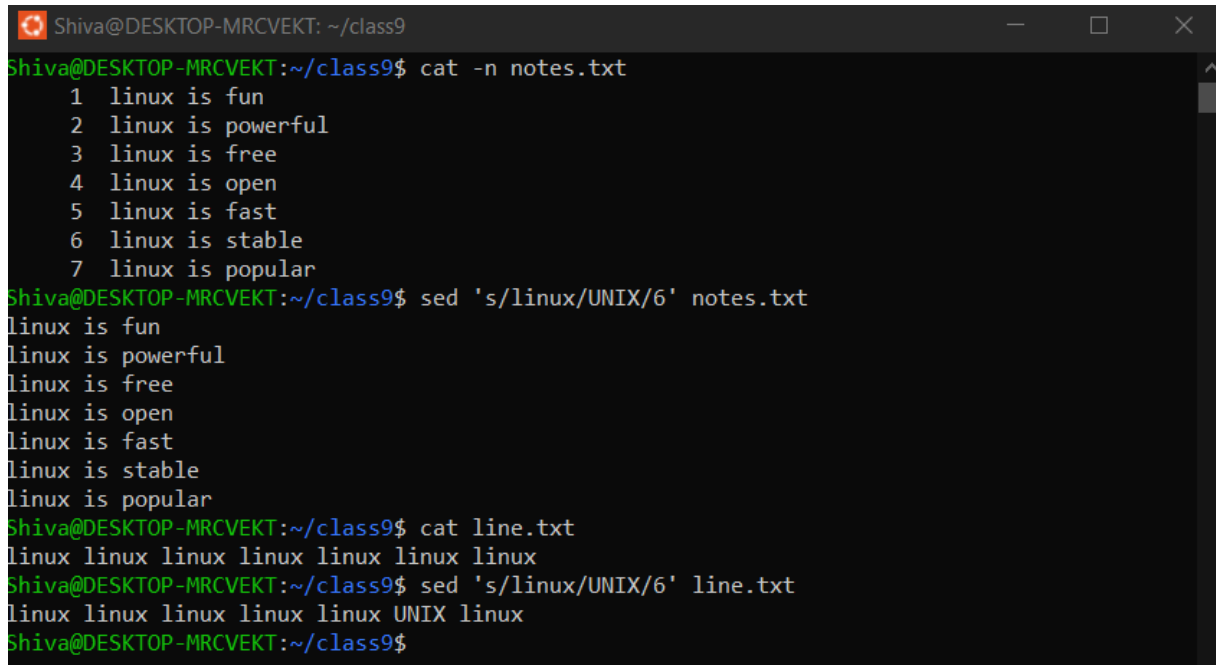
4. How to search and replace only the 6th occurrence in a whole file?

sed has a number flag for this, but it does not mean what I first thought it meant.

```
sed 's/linux/UNIX/6' notes.txt
```

When I ran this on notes.txt nothing changed at all. That is because the number counts the matches on each line separately, and every line of notes.txt has the word only once. There is no 6th match on any single line, so sed had nothing to do.

To prove the flag really works I made line.txt, which has the word seven times on one line. The same command changed only the 6th word on that line and left the other six alone.



```
Shiva@DESKTOP-MRCVEKT: ~/class9
Shiva@DESKTOP-MRCVEKT:~/class9$ cat -n notes.txt
 1 linux is fun
 2 linux is powerful
 3 linux is free
 4 linux is open
 5 linux is fast
 6 linux is stable
 7 linux is popular
Shiva@DESKTOP-MRCVEKT:~/class9$ sed 's/linux/UNIX/6' notes.txt
linux is fun
linux is powerful
linux is free
linux is open
linux is fast
linux is stable
linux is popular
Shiva@DESKTOP-MRCVEKT:~/class9$ cat line.txt
linux linux linux linux linux linux linux
Shiva@DESKTOP-MRCVEKT:~/class9$ sed 's/linux/UNIX/6' line.txt
linux linux linux linux linux UNIX linux
Shiva@DESKTOP-MRCVEKT:~/class9$
```

Screenshot 4: the /6 flag does nothing on a file with one word per line, but works on a line with seven.

So for the 6th occurrence in the whole file I need something else. In notes.txt the word appears once on each line, so the 6th occurrence is simply the one on line 6, and a number written in front of the s means a line number:

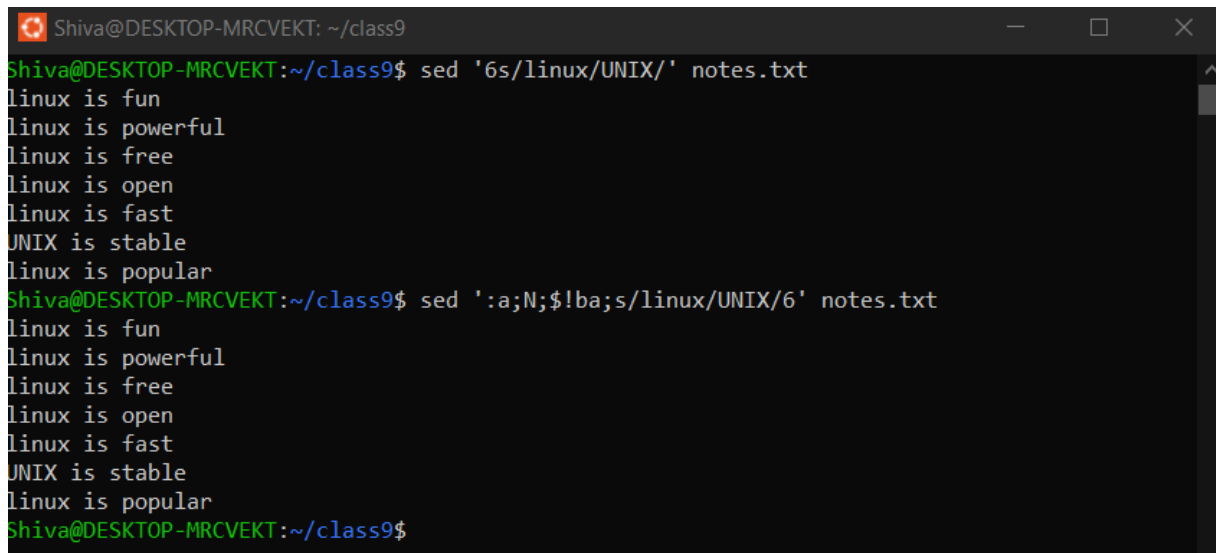
```
sed '6s/linux/UNIX/' notes.txt
```

This changed the 6th line, which became UNIX is stable.

That trick only works because my file has exactly one match per line. The way that always works is to join the whole file into one long line first, and then the 6th match really is the 6th in the file:

```
sed ':a;N;$!ba;s/linux/UNIX/6' notes.txt
```

The first part, :a;N;\$!ba, is what reads the whole file in before replacing anything. It gave the same answer, UNIX is stable on line 6, which shows both ways agree.

A terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/class9'. The first command is 'sed '6s/linux/UNIX/' notes.txt', which outputs six lines: 'linux is fun', 'linux is powerful', 'linux is free', 'linux is open', 'linux is fast', and 'UNIX is stable'. The second command is 'sed ':a;N;\$!ba;s/linux/UNIX/6' notes.txt', which outputs the same six lines. The prompt 'Shiva@DESKTOP-MRCVEKT:~/class9\$' is visible at the bottom.

Screenshot 5: the line 6 way and the whole file way give the same result.

Both of these only print the answer on the screen. To save it into the file I would add -i, and I would use -i.bak so that a copy of the old file is kept.

5. How to search and replace a case insensitive pattern in a closed file?

A closed file means a file I have not opened in vi. sed can do the whole job from the command line, so the file never has to be opened.

First, to show the problem, I ran a normal replace on mixed.txt:

```
sed 's/linux/UNIX/g' mixed.txt
```

Only the small-letter linux changed. Linux with a capital L and LINUX in all capitals were left alone, because sed matches exactly what I type.

Adding a capital I at the end fixes it:

```
sed 's/linux/UNIX/gI' mixed.txt
```

Now all three lines changed. The I stands for ignore case, so it does not care about capital or small letters. A small i also works in Ubuntu.

Those two only showed the answer on the screen. To actually change the closed file I added -i:

```
sed -i.bak 's/linux/UNIX/gI' mixed.txt
```

After that, cat mixed.txt showed the new text, so the file itself was changed without ever opening it. Writing it as -i.bak also saved the old version as mixed.txt.bak, which I could see in ls. I used the .bak form because -i on its own has no undo.

```

Shiva@DESKTOP-MRCVEKT: ~/class9
Shiva@DESKTOP-MRCVEKT:~/class9$ cat mixed.txt
linux is great
LINUX is everywhere
linux is free
Shiva@DESKTOP-MRCVEKT:~/class9$ sed 's/linux/UNIX/g' mixed.txt
linux is great
LINUX is everywhere
UNIX is free
Shiva@DESKTOP-MRCVEKT:~/class9$ sed 's/linux/UNIX/gI' mixed.txt
UNIX is great
UNIX is everywhere
UNIX is free
Shiva@DESKTOP-MRCVEKT:~/class9$ sed -i.bak 's/linux/UNIX/gI' mixed.txt
Shiva@DESKTOP-MRCVEKT:~/class9$ cat mixed.txt
UNIX is great
UNIX is everywhere
UNIX is free
Shiva@DESKTOP-MRCVEKT:~/class9$ ls
allfiles.tar.gz  employees.txt  line.txt  mixed.txt  mixed.txt.bak  notes.txt
Shiva@DESKTOP-MRCVEKT:~/class9$

```

Screenshot 6: case sensitive, then case insensitive, then saved into the file with a backup.

These are the sed forms I used in this class:

Form	What it does
s/old/new/	Changes the first match on each line.
s/old/new/g	Changes every match on each line.
s/old/new/6	Changes only the 6th match on each line.
6s/old/new/	A number in front is a line number, so this is line 6.
s/old/new/gI	Ignores capital and small letters. Capital I here.
sed -i	Saves the change into the file itself.
sed -i.bak	Saves it and keeps the old version as file.bak.

Commands I used

Command	What it does
grep "IT" employees.txt	Shows only the lines that have IT in them.
sed 's/IT/TECH/' employees.txt	Shows every line with IT changed to TECH.
awk '{print NR, \$(NF-1), \$NF}' f	Line number, then the last two columns.
tar -czvf all.tar.gz a.txt b.txt	Puts several files into one compressed archive.
tar -tzvf all.tar.gz	Lists what is inside the archive.
tar -xzvf all.tar.gz	Takes the files back out.
zip all.zip a.txt b.txt	The same idea in zip format.
sed 's/old/new/6' file	Changes the 6th match on each line.
sed '6s/old/new/' file	Changes the match on line 6.
sed ':a;N;\$!ba;s/old/new/6' file	Changes the 6th match in the whole file.
sed -i.bak 's/old/new/gI' file	Case insensitive replace, saved into the file.